PROJETO CADASTRO DE USUÁRIOS

Como construímos uma aplicação web com Spring Boot, MySQL e contêineres

Passo a passo para quem está começando em desenvolvimento Java

O que vamos aprender?

- Preparar o ambiente de desenvolvimento
- Criar e persistir o banco MySQL no Docker
- Conectar Spring Boot e MySQL
- Entender Dockerfile e compose.yml
- Executar, testar e diagnosticar o sistema

As ferramentas e suas funções

- Eclipse: editar e organizar o código Java
- Java + Spring Boot: regras de negócio e aplicação web
- Maven: dependências e empacotamento em JAR
- MySQL: armazenar os usuários
- Docker: executar a aplicação e o banco em contêineres
- Thymeleaf: mostrar os dados no navegador

Arquitetura: quem conversa com quem?

NAVEGADOR (usuarios.html)

| HTTP

v

SPRING BOOT (Controller → Service → Repository)

| JDBC / JPA

v

MYSQL (contêiner Docker)

Docker Compose inicia e conecta os serviços.

1. Preparar o ambiente

- Instalar JDK compatível com o projeto (ex.: Java 21)
- Instalar Eclipse IDE e importar o projeto Maven
- Instalar Docker Desktop e iniciar o Docker Engine
- Conferir as ferramentas no terminal:

```
java -version
mvn -version
docker --version
docker compose version
```

2. O que é o MySQL no Docker?

- MySQL é o sistema gerenciador do banco de dados
- O contêiner executa o MySQL isolado do sistema operacional
- A imagem mysql:8.4 fornece o software do banco
- O volume mantém os dados mesmo ao recriar o contêiner
- A aplicação Java acessa o banco pela rede do Compose

3. Criar o banco cadastro_usuarios

```
-- Exemplo: execução no cliente MySQL
CREATE DATABASE cadastro_usuarios
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
SHOW DATABASES;
USE cadastro_usuarios;
SHOW TABLES;
```

Duas maneiras de criar o banco

- Manual: executar CREATE DATABASE cadastro_usuarios; no MySQL
- Automática: configurar MYSQL_DATABASE no compose.yml
- O MySQL cria o banco automaticamente na primeira inicialização com volume vazio
- Se o volume já existir, mudar MYSQL_DATABASE não recria o banco

4. Estrutura no Eclipse

```
cadastro-usuarios/
├── src/main/java/com/projeto/cadastro/
│   ├── CadastroUsuariosApplication.java
│   ├── entities/Usuario.java
│   ├── repository/UsuarioRepository.java
│   ├── service/UsuarioService.java, CsvService.java
│   └── controller/UsuarioWebController.java
├── src/main/resources/templates/usuarios.html
├── src/main/resources/application.properties
├── pom.xml   ├── Dockerfile   └── compose.yml
```

5. application.properties

```
# Exemplo para Spring Boot dentro do Compose
spring.datasource.url=jdbc:mysql://mysql:3306/cadastro_usuarios
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Atenção: “mysql” é o nome do serviço Docker, não localhost.

6. Dockerfile: a imagem do Java

```
# Modelo didático: JAR já compilado pelo Maven
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
Antes de construir a imagem: mvn clean package -DskipTests
```

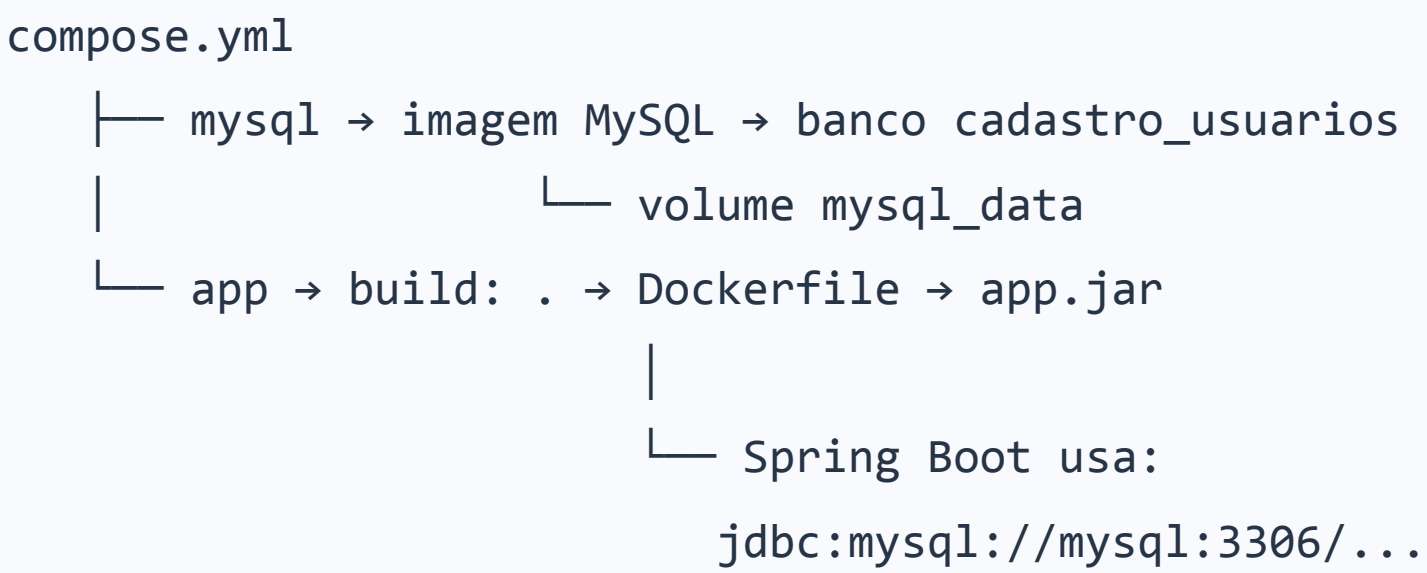
7. compose.yml: sobe tudo junto

```
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: cadastro_usuarios
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -u root -
p$$MYSQL_ROOT_PASSWORD"]
```

compose.yml: serviço da aplicação

```
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
    depends_on:
      mysql:
        condition: service_healthy
volumes:
```

8. Como os arquivos se conectam?



9. Comandos para iniciar

```
# 1. Compilar o projeto
mvn clean package -DskipTests
# 2. Criar imagens e iniciar contêineres
docker compose up --build -d
# 3. Conferir o estado
docker compose ps
# 4. Ver logs da aplicação
docker compose logs -f app
```

10. Testar a conexão MySQL

```
# Entrar no MySQL pelo contêiner
docker compose exec mysql mysql -u root -p
-- Dentro do cliente MySQL:
USE cadastro_usuarios;
SHOW TABLES;
SELECT * FROM usuarios;
SELECT COUNT(*) FROM usuarios;
```

11. Entenda as camadas do Spring

- Entity Usuario: representa os dados e a tabela
- Repository: operações de banco com JpaRepository
- Service: validações e regras do cadastro
- Controller: recebe requisições e escolhe a resposta
- Thymeleaf: apresenta formulário, tabela e contador
- CsvService: prepara os registros para exportação

12. Operações do sistema

```
GET  /usuarios                  → página e listagem
POST /usuarios/salvar            → cadastrar
POST /usuarios/atualizar/{id}  → atualizar
POST /usuarios/excluir/{id}    → excluir
GET  /usuarios/exportar-csv    → baixar CSV
O ID identifica o registro no backend,
mas não precisa aparecer na tabela HTML.
```

13. Erros comuns e como resolver

- Template não encontrado: conferir templates/usuarios.html
- Ambiguous mapping: duas rotas GET /usuarios iguais
- Connection refused: verificar MySQL, host e porta
- Access denied: conferir usuário, senha e permissões
- Mudança não aparece: recompilar o JAR e reconstruir a imagem
- Contêiner parado: inspecionar docker compose logs

14. Checklist de demonstração

- Iniciar Docker Desktop
- Executar Maven e Docker Compose
- Abrir http://localhost:8080/usuarios
- Cadastrar um usuário e verificar no MySQL
- Editar, excluir e observar o contador
- Exportar CSV e conferir os registros
- Mostrar logs e explicar cada camada

Conclusão e próximos passos

- Eclipse: escrevemos e organizamos o código
- Maven: transformamos o projeto em JAR
- Dockerfile: definimos a imagem do Spring Boot
- Compose: conectamos a aplicação ao MySQL
- MySQL: persistimos os usuários
- Próximos passos: validações, testes e segurança